Introduction to Seaborn Library

Seaborn is a **powerful Python library** for creating **beautiful and informative** data visualizations. It is built on **Matplotlib** and works seamlessly with **Pandas DataFrames**.

What You'll Learn in This Tutorial:

- Installing and Importing Seaborn
- Seaborn vs. Matplotlib (Key Differences)
- Basic Seaborn Plots (Histograms, Scatter Plots, Bar Charts)
- Advanced Seaborn Visualizations (Pair Plots, Heatmaps, Violin Plots)
- Customizing Seaborn Plots (Themes, Colors, Annotations)
- Combining Seaborn with Matplotlib

1.Installing and Importing Seaborn

Installation

To install Seaborn, use:

pip install seaborn

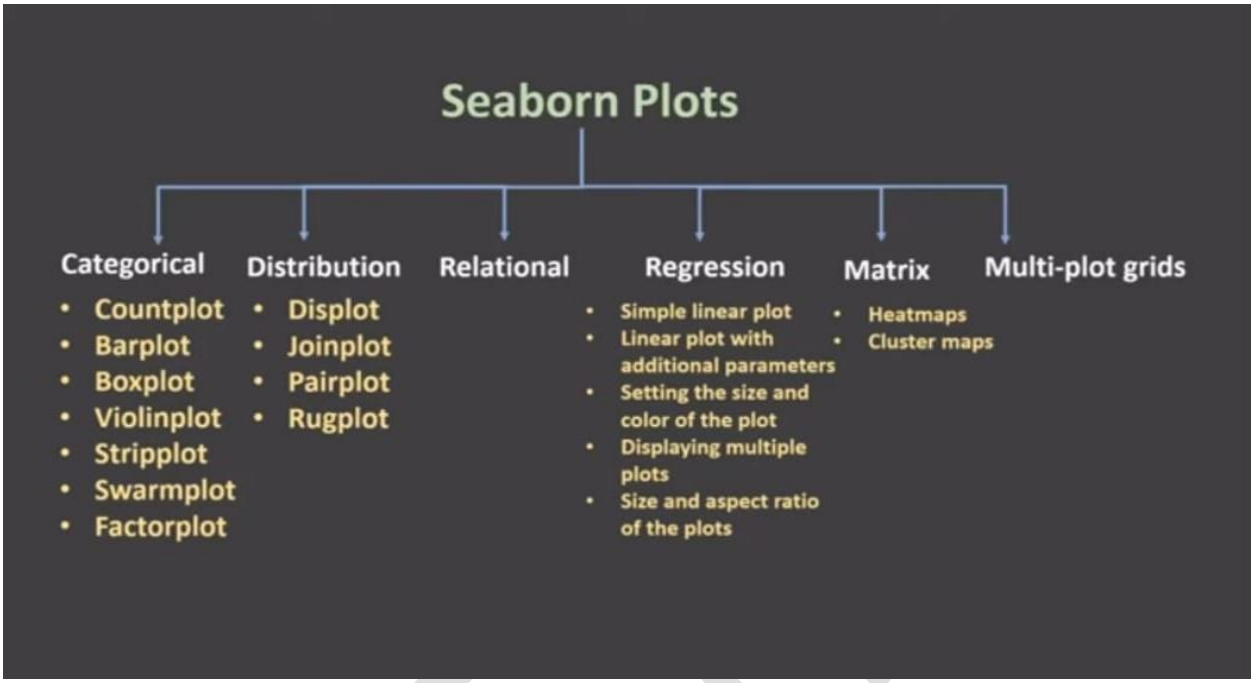
Importing Seaborn & Other Required Libraries

```
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
```

2.Seaborn vs. Matplotlib: What's the Difference?

Feature	Matplotlib 📉	Seaborn 📊
Purpose	Low-level visualization library	High-level statistical visualization
Ease of Use	Requires more code for customization	Built-in themes and aesthetics
Default Style	Basic and plain	More visually appealing
Data Handling	Works with NumPy & Pandas	Works best with Pandas DataFrames
Best for	Simple plots & highly customized visualizations	Quick and stylish statistical analysis

3.Basic Seaborn Plots



1. Line Plot

Used for visualizing trends over time.

```
# Load dataset
flights = sns.load_dataset("flights")
# Line plot
plt.figure(figsize=(8,5))
sns.lineplot(x="year", y="passengers", data=flights)
plt.title("Passenger Growth Over Time")
plt.show()
```

2. Histogram (Distribution Plot)

Used to visualize **data distribution**.

```
titanic = sns.load_dataset("titanic")
plt.figure(figsize=(8,5))
sns.histplot(titanic["age"], bins=20, kde=True, color="blue")
plt.title("Age Distribution of Titanic Passengers")
plt.show()
```

3. Scatter Plot

Used to show relationships between two numerical variables.

```
plt.figure(figsize=(8,5))
sns.scatterplot(x="age", y="fare", hue="sex", data=titanic, palette="coolwarm")
plt.title("Age vs. Fare Paid")
plt.show()
```

4. Bar Chart

Used to compare different categories.

```
plt.figure(figsize=(6,4))
sns.countplot(x="class", data=titanic, palette="viridis")
plt.title("Passenger Count by Class")
plt.show()
```

sns.barplot() → Used for Aggregated Values (Mean, Sum, etc.)

✓ Used when you want to show the **average (or another statistic) of a numerical column** grouped by a category.
✓ **Requires both `x` (category) and `y` (numerical value)**.
✓ By default, it shows the **mean** of the `y` values.

sns.countplot() → Used for Counting Categories

✓ Used when you want to show the **count (frequency) of each category** in a column.
✓ **Only needs `x` (categorical variable); no `y` is needed**.
✓ Each bar represents **how many times a category appears** in the dataset.

4.Advanced Seaborn Plots

5. Pair Plot

Shows relationships between multiple variables.

```
sns.pairplot(titanic, hue="sex", palette="husl")
plt.show()
```

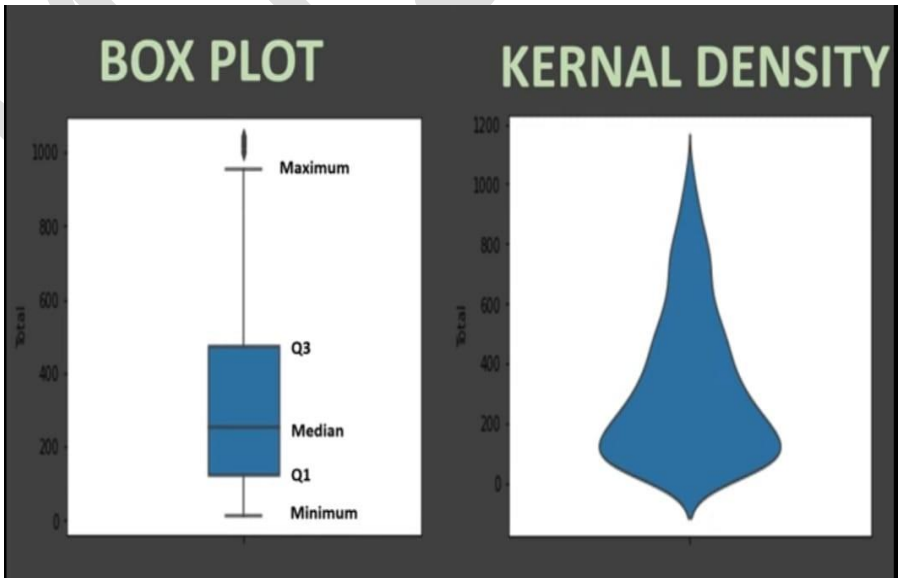
6. Box Plot

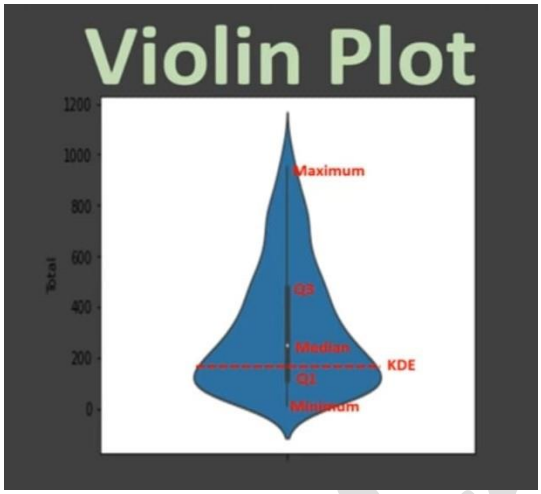
Used for visualizing outliers and distribution or statistical values.

```
plt.figure(figsize=(8,5))
sns.boxplot(x="pclass", y="age", hue="sex", data=titanic, palette="Set2")
plt.title("Age Distribution by Class & Gender")
plt.show()
```

7. Violin Plot

Combines **box plot + KDE** for distribution visualization.






```
seaborn.violinplot(*, x=None, y=None, hue=None, data=None,
order=None, hue_order=None, bw='scott', cut=2, scale='area',
scale_hue=True, gridsize=100, width=0.8, inner='box', split=False,
dodge=True, orient=None, linewidth=None, color=None, palette=None,
saturation=0.75, ax=None, **kwargs)
```

```
plt.figure(figsize=(8,5))
sns.violinplot(x="pclass", y="fare", hue="sex", split=True, data=titanic, palette="coolwarm")
plt.title("Fare Distribution by Class & Gender")
plt.show()
```

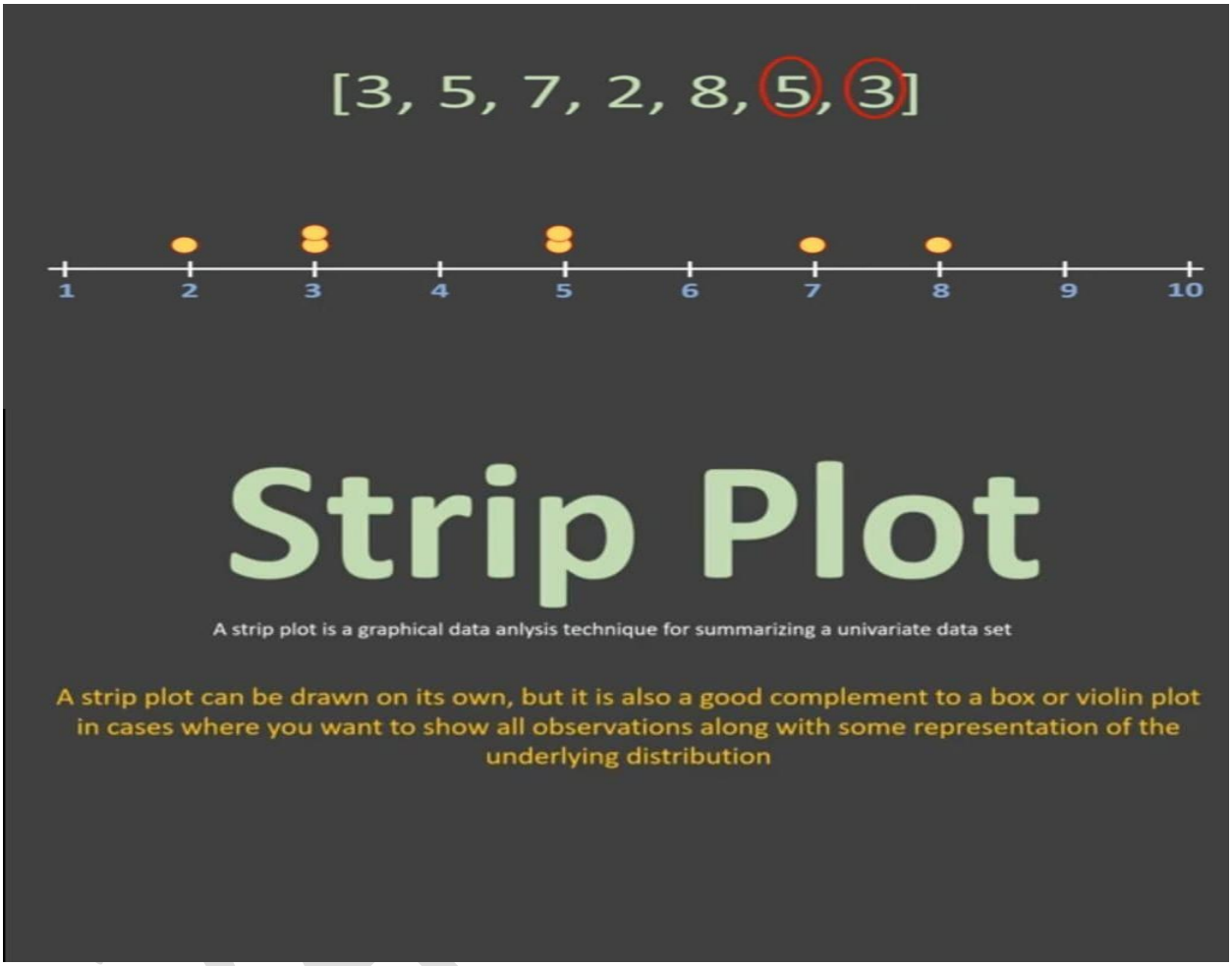
8. Heatmap (Correlation Plot)

Used to show relationships between multiple numerical variables.

```
plt.figure(figsize=(8,5))
sns.heatmap(titanic.corr(), annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Matrix")
plt.show()
```

9. Strip Plot

A **strip plot** in Seaborn is used to show the distribution of a numerical variable **against** a categorical variable. It **plots individual data points** as small dots or markers along a category, helping to see how values are spread.
Used to summarize univariate data.



Basic Syntax

```
sns.stripplot(x='category_column', y='numerical_column', data=df)
plt.show()
```

Add Colors by Category (hue)

To **color dots** based on another category:

```
sns.stripplot(x='day', y='total_bill', hue='sex', data=df, jitter=True, palette="coolwarm")
plt.show()
```

When to Use Strip Plots?

✓ **When you want to see individual data points** instead of just averages.
✓ **When a box plot or violin plot hides too much detail.**
✓ **When comparing distributions across categories.**

Combine Strip Plot with Box Plot

```
sns.boxplot(x='day', y='total_bill', data=df)
sns.stripplot(x='day', y='total_bill', data=df, color='black', jitter=True)
plt.show()
```

✓ The **box plot** shows the summary statistics.
✓ The **strip plot** shows individual data points.

10. Swarm Plot

A **swarm plot** is similar to a **strip plot**, but it arranges the dots **so they don't overlap**. This makes it easier to see **how many points** are present at each category level.

Think of it as a **better version of a strip plot** that keeps all points visible by slightly adjusting their positions.

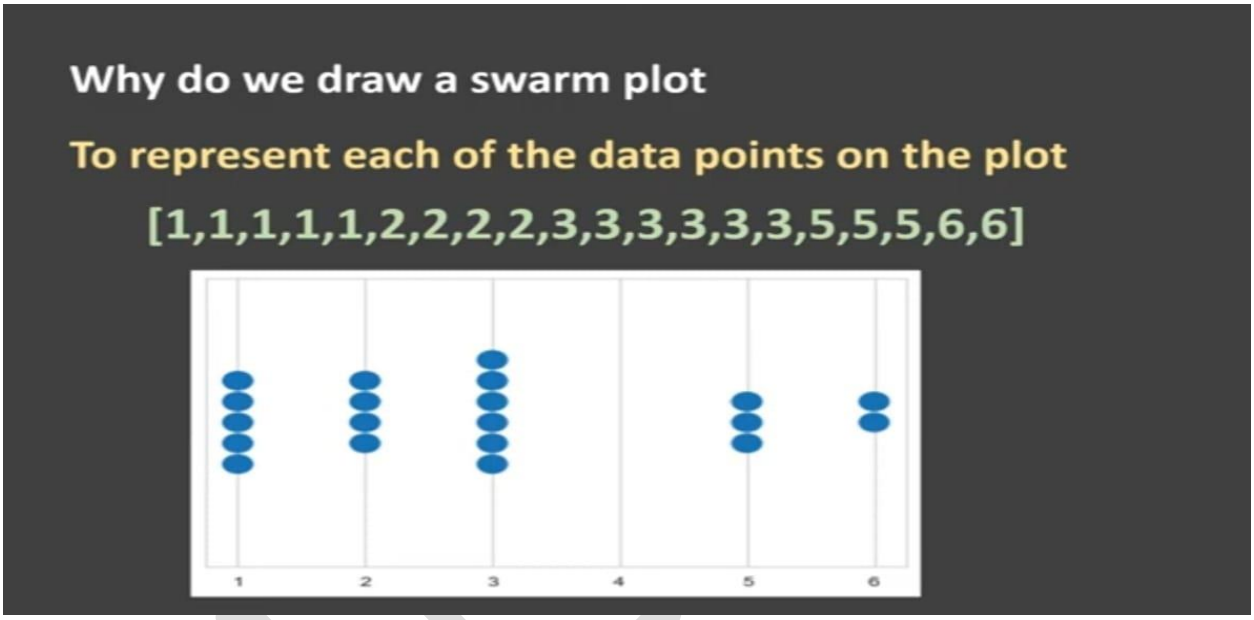
Basic Syntax

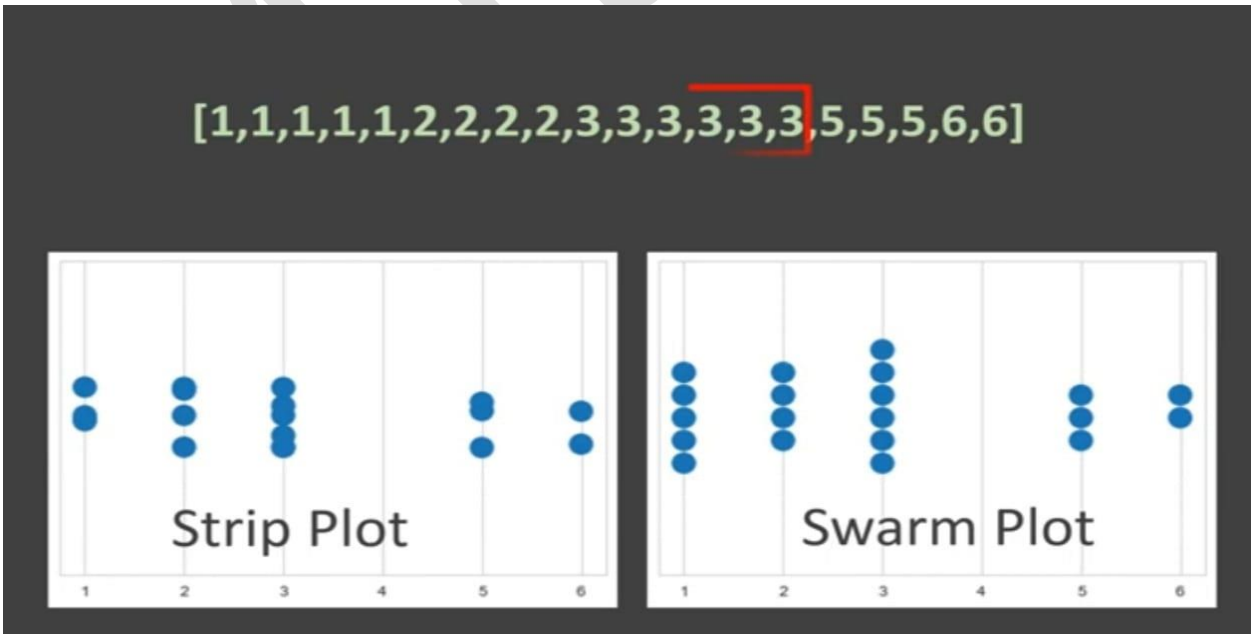
```
sns.swarmplot(x='category_column', y='numerical_column', data=df)
plt.show()
```

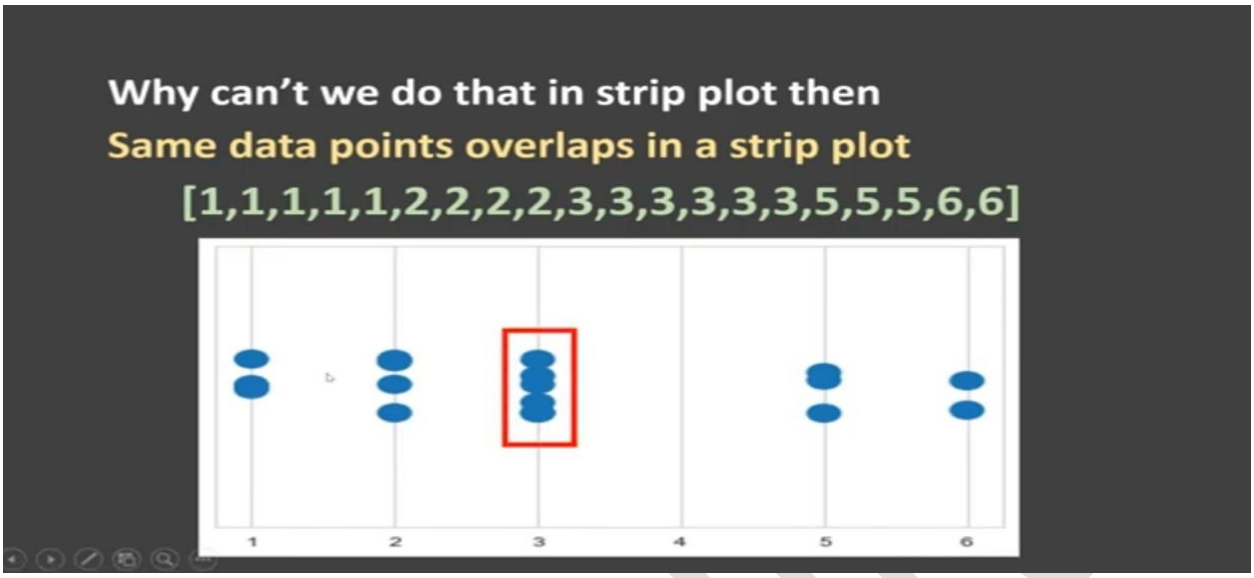
When to Use Swarm Plots?

✓ When you **want to see individual data points** clearly.
✓ When a **strip plot overlaps too much** and hides some points.
✓ When comparing **distribution patterns across categories**.

⚠ **Note: Swarm plots can be slow** for large datasets because they adjust each point's position to avoid overlap

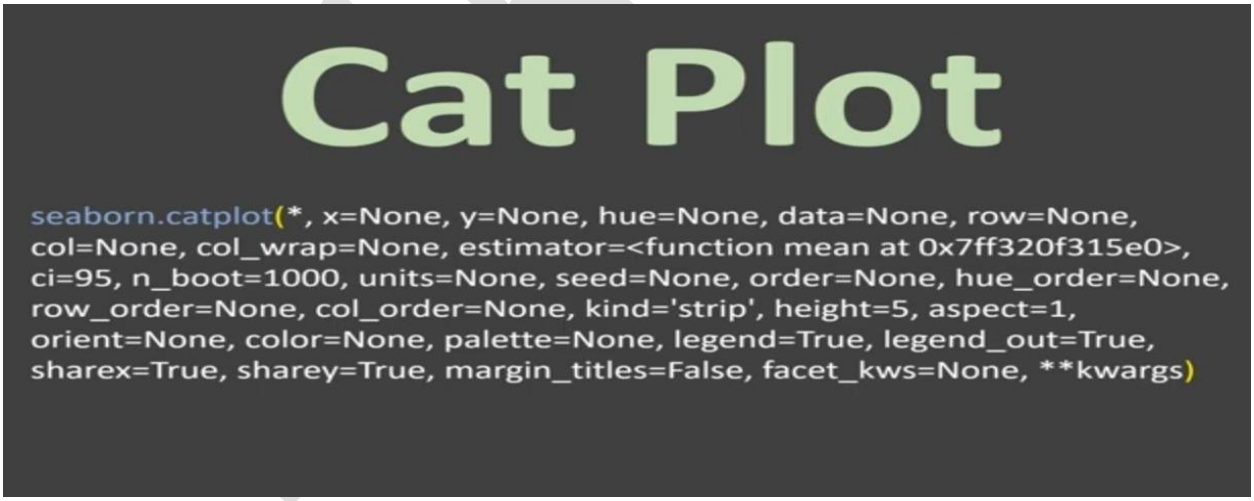






11.Factor Plot

A **Factor Plot** (now replaced by catplot in newer Seaborn versions) is a **generalized plot** that allows you to create different types of categorical plots (like bar plots, box plots, etc.) in a **single function**.



Basic Syntax

```
sns.factorplot(x='category_column', y='numerical_column', data=df, kind='bar')
plt.show()
```

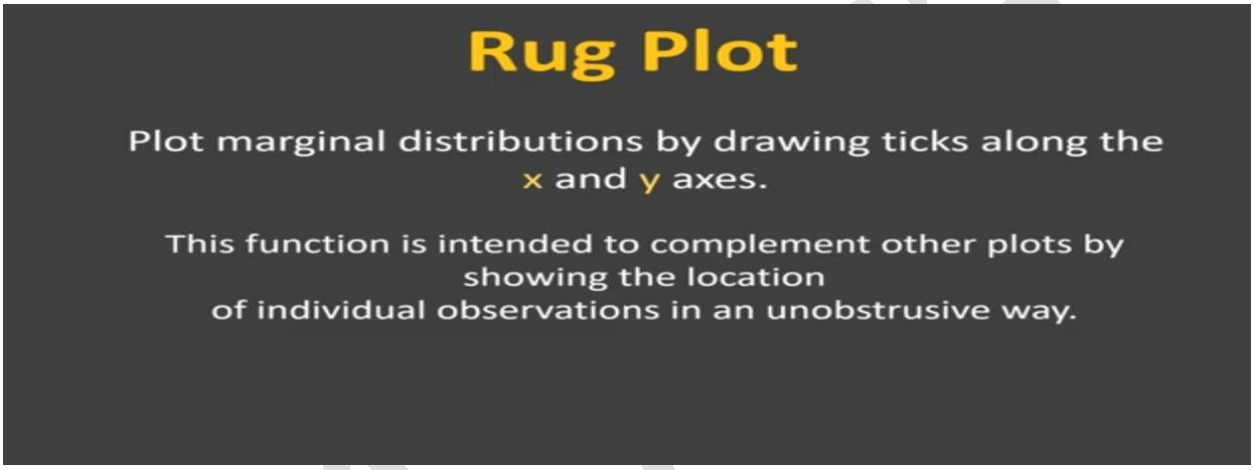
⚠ **Note:** The factorplot() function has been **replaced** by catplot() in newer Seaborn versions. Use sns.catplot() instead.

Example:

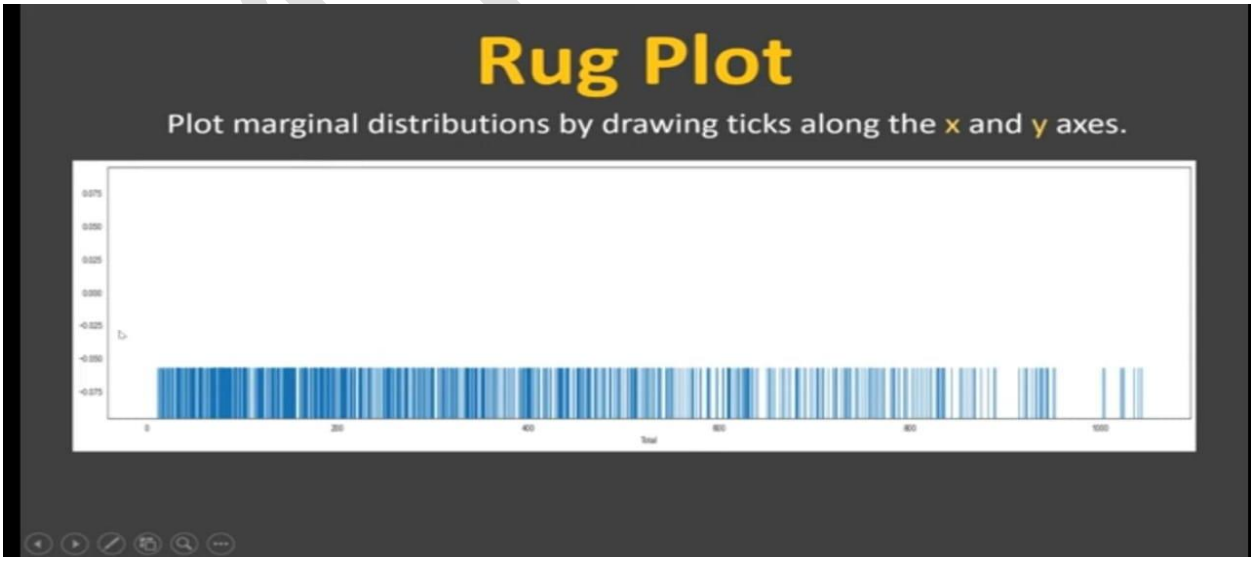
```
sns.catplot(x='day', y='total_bill', hue='sex', data=df, kind='bar', palette='coolwarm')
plt.show()
```

12.Rug Plot

A **rug plot** is a simple way to **show the distribution** of data points along an axis. It places **small ticks (lines)** at the actual data points, making it easy to see **where data is concentrated**.



💡 **Think of it as a "histogram without bins"—just showing exact data locations.**



Basic Syntax

```
sns.rugplot(x=df['total_bill'])
plt.show()
```

✓ This will place **small vertical lines** (ticks) at each data point on the x-axis.

```
seaborn.rugplot(x=None, *, height=0.025, axis=None, ax=None, data=None,
                y=None, hue=None, palette=None, hue_order=None,
                hue_norm=None, expand_margins=True, legend=True,
                a=None, **kwargs)
```

When to Use a Rug Plot?

✓ When you **want to see the exact data points** in a dataset.
✓ When you **need a simple way to visualize data concentration**.
✓ When combining with **KDE plots** for a clearer view.

⚠ Limitation

⚠ **Too many data points?** A rug plot may become cluttered.
Solution: Use a **KDE plot** or a **histogram** for a clearer view.

13.Correlation Map

A **correlation map** (also called a **correlation heatmap**) is a type of heatmap that shows the **relationship between numerical variables** in a dataset. It is created using **correlation coefficients**, which range from **-1 to 1**:

- **+1 → Perfect Positive Correlation** (Both values increase together)
- **0 → No Correlation** (No relationship)
- **-1 → Perfect Negative Correlation** (One value increases, the other decreases)

Example:

```
# Load dataset
df = sns.load_dataset("penguins")
# Compute correlation matrix
corr_matrix = df.corr(numeric_only=True)
# Create heatmap
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm", linewidths=0.5)
plt.show()
```

Difference Between a Heatmap and a Correlation Map

Feature	Heatmap 🟡🔵	Correlation Map 🔴🔵
Definition	A data visualization that represents numerical values using colors.	A specialized heatmap that shows the correlation between variables (using correlation coefficients).
Data Used	Any numerical data arranged in a **matrix or table format**.	A **correlation matrix** computed from numerical columns of a dataset.
Values	Can be **any numeric values**.	Always between **-1 and 1** (correlation values).
Use Case	To analyze **patterns, trends, and clusters** in data.	To understand **relationships between variables** (positive, negative, or no correlation).
Color Meaning	Colors represent **actual values** in the dataset.	Colors indicate **correlation strength** (red = negative, blue = positive).

When to Use Which?

- **Use a Heatmap** → When you want to **visualize data distribution** (e.g., temperature, population, sales).
- **Use a Correlation Map** → When you want to **find relationships between numerical variables** in a dataset.

14. FacetGrid

FacetGrid is a powerful tool in Seaborn that helps you create multiple plots (also called **facets**) based on different values of a categorical variable. It allows you to visualize distributions relationships, or trends**,** across different subgroups of data.

Example:

```
# Load dataset
df = sns.load_dataset("penguins")
# Create a FacetGrid with species as a categorical variable
g = sns.FacetGrid(df, col="species")  # Create grid with separate plots for each species
# Map a histogram onto each facet
g.map(sns.histplot, "flipper_length_mm")
plt.show()
```

Multiple rows and Columns:

```
g = sns.FacetGrid(df, col="sex", row="species", margin_titles=True)
g.map(sns.scatterplot, "bill_length_mm", "bill_depth_mm")
plt.show()
```

5.Customizing Seaborn Plots

1.Change Plot Style

Seaborn offers different built-in styles:

sns.set_style("whitegrid") # Options: darkgrid, whitegrid, dark, white, ticks

Change Color Palette

sns.set_palette("pastel") # Other options: deep, muted, bright, dark, colorblind

Adjust Plot Size

plt.figure(figsize=(10,6))

Add Titles and Labels

plt.title("Plot Title", fontsize=14)
plt.xlabel("X-axis Label", fontsize=12)
plt.ylabel("Y-axis Label", fontsize=12)

6.Combining Seaborn with Matplotlib

You can use Matplotlib functions to further **customize Seaborn plots**.

plt.figure(figsize=(8,5))
sns.histplot(titanic["age"], bins=20, kde=True, color="green")
# Custom Matplotlib styling
plt.axvline(titanic["age"].mean(), color='red', linestyle="dashed", label="Mean Age")
plt.legend()
plt.show()

Experimenting Different Data Sets

1. Load a Different Dataset

We'll use the **"penguins"** dataset, which contains data on different penguin species.

```
penguins = sns.load_dataset("penguins")  # Load dataset
print(penguins.head())  # Show first few rows
```

2. Custom Scatter Plot with Colors and Annotations

We'll compare **bill length and bill depth** for different penguin species:

```
plt.figure(figsize=(8,5))  # Set figure size
sns.set_style("whitegrid")  # Apply style
sns.scatterplot(
    x="bill_length_mm",
    y="bill_depth_mm",
    hue="species",  # Different colors for species
    style="species",  # Different markers for species
    palette="Dark2",  # Custom color palette
    data=penguins
)
plt.title("Penguin Bill Length vs Bill Depth", fontsize=14)  # Add title
plt.xlabel("Bill Length (mm)", fontsize=12)  # X-axis label
plt.ylabel("Bill Depth (mm)", fontsize=12)  # Y-axis label
plt.legend(title="Penguin Species")  # Custom legend title
plt.show()
```

3. Box Plot with Custom Colors

We'll compare **flipper lengths** across species:

```
plt.figure(figsize=(8,5))
sns.boxplot(
    x="species",
    y="flipper_length_mm",
    data=penguins,
    palette="Set2"
)

plt.title("Flipper Length by Penguin Species", fontsize=14)
plt.xlabel("Species", fontsize=12)
```

```
plt.ylabel("Flipper Length (mm)", fontsize=12)
plt.show()
```

4. Violin Plot with Hue for Sex

We'll visualize **bill length distribution for male and female penguins**:

```
plt.figure(figsize=(8,5))
sns.violinplot(
    x="species",
    y="bill_length_mm",
    hue="sex",
    split=True,
    palette="coolwarm",
    data=penguins
)

plt.title("Bill Length by Species and Sex", fontsize=14)
plt.xlabel("Species", fontsize=12)
plt.ylabel("Bill Length (mm)", fontsize=12)
plt.legend(title="Sex")
plt.show()
```

5. Heatmap for Correlation Between Numeric Features

Let's find relationships between different measurements:

```
plt.figure(figsize=(8,5))
corr = penguins.corr()  # Calculate correlation matrix
sns.heatmap(corr, annot=True, cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Between Features", fontsize=14)
plt.show()
```

1. Load the Titanic Dataset 

We'll use the **Titanic dataset**, which contains information on passengers and their survival.

```
titanic = sns.load_dataset("titanic")
print(titanic.head())  # Show first few rows
```

2. Customized Bar Plot with Annotations

We'll show **survival rates by class** with annotations!

```
plt.figure(figsize=(8,5))
sns.set_style("darkgrid")
ax = sns.barplot(
    x="class",
    y="survived",
    data=titanic,
    palette="viridis"
)

# Add annotations
for p in ax.patches:
    ax.annotate(f"{p.get_height():.2f}",
                (p.get_x() + p.get_width() / 2, p.get_height()),
                ha="center", va="bottom", fontsize=12)

plt.title("Survival Rate by Passenger Class", fontsize=14)
plt.xlabel("Passenger Class", fontsize=12)
plt.ylabel("Survival Rate", fontsize=12)
plt.show()
```

3. Swarm Plot with Custom Colors 

Swarm plots show **data distribution** while avoiding overlap.

```
plt.figure(figsize=(8,5))
sns.set_style("white")
sns.swarmplot(
    x="class",
    y="age",
    hue="sex",
    palette="coolwarm",
    data=titanic
)
```

```
plt.title("Age Distribution by Class & Gender", fontsize=14)
plt.xlabel("Passenger Class", fontsize=12)
plt.ylabel("Age", fontsize=12)
plt.legend(title="Sex")
plt.show()
```

4. KDE Plot (Density Estimation) for Age Distribution

Let's see the **age distribution** of survivors vs non-survivors.

```
plt.figure(figsize=(8,5))
sns.kdeplot(
   x=titanic["age"],
   hue=titanic["survived"],
   fill=True,
   palette=["red", "green"],
   alpha=0.5
)

plt.title("Age Distribution of Survivors vs Non-Survivors", fontsize=14)
plt.xlabel("Age", fontsize=12)
plt.ylabel("Density", fontsize=12)
plt.legend(title="Survived", labels=["No", "Yes"])
plt.show()
```

5. Pair Plot for Multi-Feature Analysis

Pair plots show **relationships between multiple numerical columns**.

```
sns.pairplot(
   titanic,
   hue="survived",
   palette="coolwarm",
   vars=["age", "fare", "pclass"]  # Selecting important columns
)
plt.show()
```

6. Heatmap of Missing Values

Let's visualize **which data is missing** in the Titanic dataset.

```
plt.figure(figsize=(8,5))
sns.heatmap(titanic.isnull(), cmap="Reds", cbar=False)
plt.title("Missing Data in Titanic Dataset", fontsize=14)
plt.show()
```

Let's explore advanced visualizations and customization techniques in Seaborn

We'll cover:

1. **FacetGrid for multi-panel plots**
2. **JointPlot for relationships between two variables**
3. **Violin and Boxen Plots for better distributions**
4. **Advanced Heatmaps with Clustering**
5. **Customizing styles, themes, and legends**

1. FacetGrid – Multi-Panel Plots 🖼️

FacetGrid is useful for visualizing **distributions across multiple categories**.

```
g = sns.FacetGrid(titanic, col="sex", row="pclass", margin_titles=True)
g.map_dataframe(sns.histplot, x="age", bins=20, color="purple")
plt.show()
```

Explanation:

- Creates a **grid of plots** based on **sex** and **class**.
- Uses **histogram (histplot)** to show **age distribution**.

2. JointPlot – Relationship Between Two Variables 🖇️

Joint plots show **scatter + histogram + density** together.

```
sns.jointplot(

data=titanic,
    x="age",
    y="fare",
    hue="sex",
    kind="kde",
    fill=True,
    palette="coolwarm"
)
plt.show()
```

Explanation:

- Shows **relationship between Age & Fare**.
- Uses **Kernel Density Estimation (KDE)**.
- Different **colors for male & female**.

3. Violin & Boxen Plots – Advanced Distributions 🎻

Violin & Boxen plots help understand **data spread & outliers**.

```
plt.figure(figsize=(8,5))
sns.violinplot(
    x="pclass",
    y="fare",
    hue="sex",
    split=True,
    palette="muted",
    data=titanic
)

plt.title("Fare Distribution by Class & Gender", fontsize=14)
plt.xlabel("Passenger Class", fontsize=12)
plt.ylabel("Fare", fontsize=12)
plt.legend(title="Sex")
plt.show()
```

Explanation:

- **Violin plots** combine **box plot + KDE**.
- Shows **distribution of fares per class & gender**.

4. Advanced Heatmap – Clustering with Correlation Matrix 🔥

We'll **cluster features** based on **correlation values**.

```
plt.figure(figsize=(8,5))
corr = titanic.corr()
sns.clustermap(
    corr,
    cmap="coolwarm",
    annot=True,
    linewidths=0.5,
    figsize=(8,5)
)

plt.title("Clustered Correlation Heatmap", fontsize=14)
plt.show()
```

Explanation:

- Uses **clustermap** to group **similar features together**.
- Helps identify **strong relationships** between variables.

Example Comparison: Matplotlib vs. Seaborn

Let's compare **Matplotlib and Seaborn** by plotting the same dataset.

1. Matplotlib (Manual Styling)

```
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
# Load dataset
titanic = sns.load_dataset("titanic")
# Matplotlib Plot
plt.figure(figsize=(6,4))
plt.bar(titanic["class"].value_counts().index, titanic["class"].value_counts().values, color="blue")
plt.xlabel("Passenger Class")
plt.ylabel("Count")
plt.title("Passenger Count by Class (Matplotlib)")
plt.show()
```

2. Seaborn (Simpler & Prettier)

```
sns.set_style("darkgrid")
plt.figure(figsize=(6,4))
sns.countplot(x="class", data=titanic, palette="coolwarm")
plt.title("Passenger Count by Class (Seaborn)")
plt.show()
```

Key Differences:

1. **Seaborn** automatically applies colors & styles.
2. **Matplotlib** requires manual customization.

When to Use Seaborn vs. Matplotlib?

✅ **Use Matplotlib** if you need **highly customized plots**.
✅ **Use Seaborn** if you want **quick, stylish, and statistical plots**.